Interfaces and Comparable

why one sort method works on everything

The problem

We want to write `sort` **once**, and use it on:

- `Strings`, like `"S"` and `"O"`
- `Doubles`, like `0.42` and `0.17`
- things that don't exist yet, like a `Song` class you write yourself

A sort needs exactly **one thing** from its items: a way to ask **"which of these two comes first?"**

An interface is a promise

```
1  // Built into Java (simplified):
2  public interface Comparable<T> {
3      int compareTo(T that);
4  }
```

An **interface** is a list of methods.

A class that **implements** it promises to have those methods.

`Comparable` lists just **one**: `compareTo`.

Think of it as a job posting with one required skill: any class that has the skill can do the job.

What compareTo returns

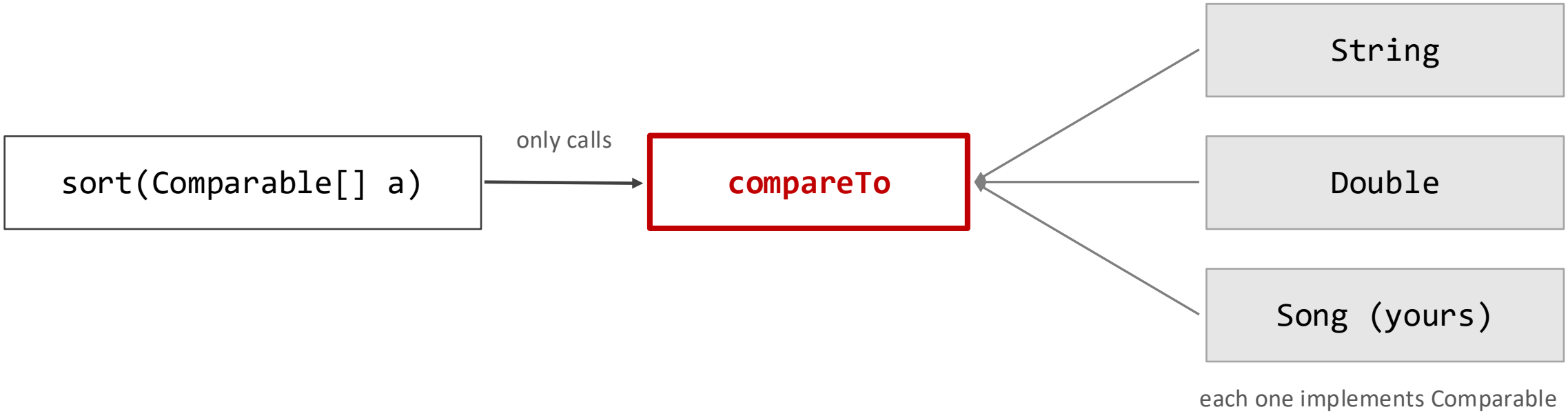
v.compareTo(w) returns...	which means
a negative number	v comes before w
zero	v and w are equal
a positive number	v comes after w

Only the **sign** matters, not the size of the number.

Our `less` helper turns that into a yes-or-no question:

```
1  private static boolean less(Comparable v, Comparable w) {
2      return v.compareTo(w) < 0;     // negative: v comes first
3  }
```

Why it matters: sort only knows the promise



Inside `sort`, Java doesn't know or care **what** the items are. It only knows they kept the promise: they have `compareTo`.

`String` and `Double` already implement `Comparable`. That's why the same `sort` works on both.

Your own class can keep the promise too

```
1  public class Song implements Comparable<Song> {
2      private final String title;
3      private final int plays;
4
5      // ... constructor and toString ...
6
7      // Songs with fewer plays come first.
8      public int compareTo(Song that) {
9          return Integer.compare(this.plays, that.plays);
10     }
11 }
```

`implements Comparable<Song>` **makes the promise**.

`compareTo` **keeps the promise**.

`Integer.compare(x, y)` returns negative, zero, or positive for you.

Now insertion sort, **unchanged**, sorts songs:

```
[Rain (12), Echo (95), Blue (340)]
```

What's <Song>**?** It says what a Song gets compared to: another Song. That's why `compareTo` takes a Song. Our sorts write plain `Comparable`, without the `<...>`. That's the book's shortcut, and it's where the "unchecked" compiler note comes from (see next slide).

Things that trip people up

1. Leave out `implements Comparable`, and **Java won't let you sort it:**

```
error: incompatible types: Plain[] cannot be converted to Comparable[]
```

2. `int` and `double` (lowercase) **can't be sorted this way.** They're primitive types, not classes, so they can't implement interfaces. Use `Integer` and `Double` (capital letters) instead. That's why `SortClient` uses a `Double[]`.

3. The "unchecked or unsafe operations" note when you compile **is expected.** It comes from using `Comparable` the same way the book does. It's a note, not an error.

The short version

An **interface** is a promise to have certain methods.

`Comparable` promises **one method**: `compareTo`.

Our sorts only ever call `compareTo` (through `less`), so they work on **anything that keeps the promise**: letters, numbers, or classes you write yourself.